

15 - Introducción al DOM

Introducción al DOM: ¿Qué es y por qué es tan importante?

El DOM (Document Object Model, o Modelo de Objetos del Documento) es una interfaz que representa la estructura de un documento HTML o XML. Piensa en el DOM como un puente entre el código HTML que ves en tu archivo y el navegador, que permite interactuar con los elementos de una página web de manera dinámica utilizando JavaScript.

Imagina que el DOM es un árbol gigante, donde cada etiqueta HTML es una rama o una hoja. Desde JavaScript, podemos acceder a cualquier parte de ese árbol, mover ramas, cambiar hojas o incluso cortar partes del árbol y volverlas a pegar. Esto es lo que hace que las páginas web puedan ser dinámicas e interactivas.

¿Por qué es tan importante el DOM?

El DOM es fundamental porque:

- Permite manipular elementos HTML: Con JavaScript, puedes cambiar texto, estilos, agregar o eliminar elementos, cambiar imágenes, etc., sin necesidad de recargar toda la página.
- Facilita la interacción con el usuario: Cada vez que ves un botón que cambia su color al hacer clic o un mensaje que aparece de repente, es el DOM en acción.
- Crea experiencias dinámicas: Sin el DOM, las páginas web serían estáticas, lo que significa que no podrían adaptarse ni responder a las acciones del usuario.

¿Cómo funciona el DOM?

Cada elemento de una página HTML se convierte en un nodo dentro del árbol del DOM. Estos nodos incluyen:

- Nodos de elemento: Representan las etiquetas HTML como <div>, <p>, <h1>, etc.
- Nodos de texto: Son los contenidos de texto dentro de las etiquetas.
- Nodos de atributo: Representan los atributos de las etiquetas, como class, id, etc.

Cuando cargas una página web, el navegador crea el DOM a partir del archivo HTML. Este DOM es lo que manipulamos con JavaScript.

Ejemplo básico de la estructura del DOM

Supongamos que tenemos un código HTML simple como este:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Mi primera página</title>
  </head>
  <body>
    <h1>¡Hola Mundo!</h1>
    <p>Este es un párrafo.</p>
    <button>Haz clic aquí</button>
  </body>
</html>
```

En el DOM, esta estructura HTML se ve así:

```
- html
  - head
    - title
    - "Mi primera página"
  - body
    - h1
    - "¡Hola Mundo!"
    - p
    - "Este es un párrafo."
    - button
    - "Haz clic aquí"
```

Manipulación del DOM con JavaScript

Ahora que sabes qué es el DOM, veamos cómo podemos interactuar con él. Vamos a realizar algunas operaciones básicas para manipular el contenido y los elementos de una página utilizando JavaScript.

1. Acceder a elementos del DOM

Para empezar a manipular el DOM, primero necesitamos seleccionar o acceder a los elementos HTML que queremos modificar. Para ello, JavaScript nos proporciona métodos como:

- `document.getElementById()`: Selecciona un elemento por su atributo id.
- `document.getElementsByClassName()`: Selecciona todos los elementos que tienen una clase en particular.
- `document.querySelector()`: Selecciona el primer elemento que coincide con un selector CSS.

Ejemplo: Seleccionar y modificar el contenido de un elemento con `id="titulo"`.

```
<h1 id="titulo">¡Hola Mundo!</h1>
```

```
<script>
  let titulo = document.getElementById("titulo");
  titulo.textContent = "¡Hola, JavaScript!";
</script>
```

Aquí estamos accediendo al elemento `<h1>` con `id="titulo"` y cambiando su texto a "¡Hola, JavaScript!".

2. Modificar estilos de un elemento

También puedes cambiar los estilos de un elemento directamente desde JavaScript usando la propiedad `style`.

Ejemplo: Cambiar el color del texto de un párrafo.

```
<p id="parrafo">Este es un párrafo.</p>

<script>
  let parrafo = document.getElementById("parrafo");
  parrafo.style.color = "blue"; // Cambia el color del texto a azul
  parrafo.style.fontSize = "20px"; // Cambia el tamaño de la fuente
</script>
```

3. Agregar o eliminar elementos

Podemos usar JavaScript para crear nuevos elementos o eliminarlos del DOM.

Ejemplo: Agregar un nuevo párrafo.

```
<div id="contenido"></div>

<script>
  let contenido = document.getElementById("contenido");

  // Crear un nuevo párrafo
  let nuevoParrafo = document.createElement("p");
  nuevoParrafo.textContent = "Este es un nuevo párrafo creado
dinámicamente.";

  // Añadir el párrafo al div
  contenido.appendChild(nuevoParrafo);
</script>
```

En este ejemplo, hemos creado un nuevo elemento <p>, le hemos dado un contenido de texto y lo hemos agregado a un contenedor existente en la página.

4. Eventos en el DOM

Los eventos del DOM (Document Object Model) en JavaScript son mecanismos fundamentales que permiten interactuar con los elementos de una página web en respuesta a diferentes acciones del usuario o del sistema. Los eventos reflejan las interacciones que ocurren en la página, como hacer clic en un botón, mover el ratón sobre un elemento, presionar una tecla, cargar la página, entre otros. Los desarrolladores utilizan eventos para construir interfaces interactivas y dinámicas, controlando cómo la página responde a diversas acciones.

¿Qué es un evento?

Un evento es cualquier cosa que ocurra en el navegador y que el navegador puede "escuchar" o detectar. Por ejemplo, cuando un usuario hace clic en un botón, se dispara un evento de clic (click), y JavaScript puede "escuchar" este evento para ejecutar algún código en respuesta. Los eventos pueden ser

generados por el usuario (como un clic o una pulsación de tecla) o por el navegador (como la carga de la página o el fin de una animación).

Tipos de eventos más comunes

Eventos de ratón: Estos eventos se disparan cuando el usuario interactúa con el ratón o dispositivos apuntadores.

- **click:** Ocurre cuando se hace clic en un elemento.
- **dblclick:** Ocurre cuando se hace doble clic en un elemento.
- **mouseover:** Cuando el ratón pasa sobre un elemento.
- **mouseout:** Cuando el ratón sale de un elemento.
- **mousemove:** Se activa cuando el ratón se mueve dentro de un elemento.
- **mousedown:** Cuando el botón del ratón se presiona sobre un elemento.
- **mouseup:** Cuando el botón del ratón se suelta después de haber sido presionado.

Ejemplo de uso:

```
const button = document.getElementById('myButton');
button.addEventListener('click', () => {
    alert('¡Hiciste clic en el botón!');
});
```

Eventos de teclado: Estos eventos permiten capturar interacciones relacionadas con el teclado.

- **keydown:** Se dispara cuando se presiona una tecla.
- **keyup:** Se dispara cuando se suelta una tecla.
- **keypress:** Ocurre mientras se mantiene presionada una tecla. (Es obsoleto en algunos navegadores).

Ejemplo de uso:

```
document.addEventListener('keydown', (event) => {
  console.log(`Tecla presionada: ${event.key}`);
});
```

Eventos de formulario: Los eventos de formulario son esenciales para capturar interacciones con los campos de entrada y los formularios.

- submit: Se activa cuando un formulario es enviado.
- change: Ocurre cuando el valor de un campo de entrada cambia.
- input: Se dispara cada vez que el valor de un campo de entrada cambia, incluso durante la escritura.
- focus: Se activa cuando un campo de entrada recibe el foco.
- blur: Ocurre cuando un campo de entrada pierde el foco.

Ejemplo de uso:

```
const form = document.getElementById('myForm');
form.addEventListener('submit', (event) => {
  event.preventDefault(); // Evitar el envío del formulario
  alert('Formulario enviado!');
});
```

Eventos de ventana: Estos eventos están asociados con la ventana del navegador.

- load: Se dispara cuando la página y todos los recursos (imágenes, scripts) han cargado completamente.
- resize: Ocurre cuando la ventana del navegador cambia de tamaño.
- scroll: Se dispara cuando se hace scroll en la página.
- unload: Ocurre cuando el usuario abandona la página.

Ejemplo de uso:

```
window.addEventListener('resize', () => {  
    console.log('La ventana ha cambiado de tamaño.');
```

Eventos de documentos: Estos eventos permiten controlar la interacción del usuario con el documento completo.

- DOMContentLoaded: Se activa cuando el HTML se ha cargado y procesado, sin esperar a que los estilos y recursos externos estén listos.
- readyStateChange: Se dispara cuando cambia el estado del documento (cargando, interactivo o completo).

Ejemplo de uso:

```
document.addEventListener('DOMContentLoaded', () => {  
    console.log('El documento está listo.');
```

Los eventos en el DOM permiten crear experiencias de usuario ricas y reactivas, capturando cualquier interacción con la página y ejecutando código específico en respuesta. Entender cómo funcionan los eventos y cómo manipularlos de manera eficiente es esencial para desarrollar aplicaciones interactivas con JavaScript.

Conclusión:

El DOM permite que nuestras páginas web sean dinámicas e interactivas. Con él, podemos:

- Cambiar el contenido o el estilo de una página sin necesidad de recargarla.
- Crear experiencias interactivas respondiendo a eventos como clics o desplazamientos.
- Manipular la estructura de la página, agregando, eliminando o modificando elementos.

Y lo mejor de todo, es que lo hacemos utilizando JavaScript, lo que nos da un control completo sobre cómo se comporta la página en tiempo real.

El DOM es el motor que hace posible la magia detrás de la interacción en la web, y al dominar su uso, puedes llevar tus páginas al siguiente nivel de interactividad y funcionalidad. ¡Es el verdadero superpoder de un desarrollador web!

Storage y JSON

En el desarrollo de aplicaciones web modernas, es fundamental poder manejar y persistir información en el navegador para mejorar la experiencia del usuario y optimizar el rendimiento de la aplicación. Para ello, contamos con herramientas como Local Storage, Session Storage, y el formato JSON. Estas tecnologías nos permiten almacenar datos temporal o permanentemente en el lado del cliente, sin necesidad de recurrir a un servidor externo para gestionar esta información. A continuación, vamos a profundizar en qué son y para qué usamos Local Storage, Session Storage y JSON.

¿Por qué usamos Local Storage y Session Storage?

Local Storage

El Local Storage es una herramienta que permite almacenar información en el navegador de forma persistente, es decir, los datos guardados permanecen incluso después de que el usuario cierre la ventana o pestaña del navegador. Esto es útil cuando queremos que la página "recuerde" ciertas preferencias o configuraciones del usuario sin necesidad de guardar esos datos en una base de datos externa. Al utilizar Local Storage, podemos optimizar el rendimiento de la

aplicación al reducir la dependencia del servidor para almacenar información que no es crítica.

Ejemplos de uso:

- Guardar el tema preferido por el usuario (oscuro o claro).
- Recordar el nombre del usuario o su idioma preferido para futuras visitas.
- Almacenar productos agregados al carrito de compras de una tienda online.

Session Storage

El Session Storage, por otro lado, es adecuado para guardar datos temporales que solo deben durar durante la sesión actual del usuario. Los datos almacenados en Session Storage se eliminan automáticamente cuando el usuario cierra la pestaña o ventana del navegador. Esta característica lo hace ideal para casos en los que no necesitamos que los datos persistan más allá de una sesión de navegación.

Ejemplos de uso:

- Mantener el estado de un formulario que el usuario está completando, evitando perder datos si refresca la página.
- Almacenar temporalmente el estado de autenticación de un usuario durante su sesión en un sitio web.

JSON

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos utilizado principalmente en aplicaciones web para transmitir información entre el cliente (navegador) y el servidor. Su estructura se basa en pares clave-valor, lo que lo hace fácil de leer y escribir tanto para humanos como para máquinas. Aunque su nombre proviene de JavaScript, JSON es independiente del lenguaje de programación y es compatible con muchos lenguajes, como Python, Java, PHP, entre otros.

Estructura Básica de JSON

JSON tiene dos estructuras principales:

Objetos: Representados por llaves {}, contienen pares clave-valor.

```
{  
  "nombre": "Juan",  
  "edad": 30,  
  "esEstudiante": false  
}
```

Arreglos (Arrays): Representados por corchetes [], contienen una lista de valores.

```
[  
  "manzana",  
  "naranja",  
  "plátano"  
]
```

¿Por qué usamos JSON?

Facilita la comunicación entre cliente y servidor: JSON es el formato más común en las APIs para enviar y recibir datos. Por ejemplo, si una aplicación necesita mostrar un listado de productos, el servidor enviará esos datos en formato JSON para que el frontend los muestre.

```
{
  "productos": [
    {"nombre": "Laptop", "precio": 1000},
    {"nombre": "Teléfono", "precio": 500}
  ]
}
```

Almacenamiento en el navegador: JSON permite convertir objetos complejos de JavaScript en texto para almacenarlos en LocalStorage o SessionStorage. Esto es crucial porque estos tipos de almacenamiento solo aceptan cadenas de texto.

Por ejemplo, para guardar un objeto en LocalStorage:

```
const usuario = { nombre: "Ana", edad: 25 };
localStorage.setItem('usuario', JSON.stringify(usuario)); // Lo
convierte a cadena JSON
```

Y para recuperarlo:

```
const usuarioGuardado =
JSON.parse(localStorage.getItem('usuario'));
```

Ventajas del JSON

- **Ligero y rápido:** JSON es más ligero que otros formatos de intercambio de datos, como XML.

- **Fácil de leer:** Su formato es simple y su sintaxis es familiar para los programadores de JavaScript.
- **Versatilidad:** Al ser independiente del lenguaje, se puede usar en aplicaciones web, APIs, bases de datos NoSQL como MongoDB, y más.

JSON es esencial en el desarrollo web moderno porque permite una fácil manipulación y transmisión de datos de manera eficiente y estructurada.

¿Por qué usamos JSON en conjunto con Local Storage y Session Storage?

Tanto Local Storage como Session Storage solo permiten almacenar cadenas de texto. Sin embargo, en muchas aplicaciones necesitamos manejar datos más complejos, como objetos y arrays. Aquí es donde entra en juego JSON (JavaScript Object Notation). JSON es un formato que permite transformar objetos JavaScript y estructuras de datos complejas en cadenas de texto mediante el uso de `JSON.stringify()`. De igual forma, al recuperar estos datos, podemos convertirlos nuevamente a su forma original con `JSON.parse()`.

¿Para qué usamos JSON?

1. *Almacenamiento de datos estructurados:* Gracias a JSON, podemos guardar objetos complejos y datos estructurados en Local Storage o Session Storage. Esto es fundamental para el almacenamiento eficiente de datos como configuraciones de usuario, detalles de productos, o listas de elementos.

2. *Comunicación con APIs:* Además de su uso en el almacenamiento local, JSON es el formato estándar para la comunicación entre el frontend y el backend en aplicaciones web. Cuando intercambiamos datos con un servidor a través de una API, normalmente usamos JSON para serializar y deserializar los datos, facilitando la interoperabilidad entre sistemas.

Ejemplo práctico de almacenamiento usando JSON:

```
const usuario = {
  nombre: "Ana",
  edad: 30,
  preferencias: {
    tema: "oscuro",
    idioma: "es"
  }
};

// Guardar en Local Storage
localStorage.setItem("usuario", JSON.stringify(usuario));

// Recuperar del Local Storage
const usuarioGuardado =
JSON.parse(localStorage.getItem("usuario"));
console.log(usuarioGuardado.preferencias.tema); // "oscuro"
```

Beneficios de usar Local Storage, Session Storage y JSON

Mejorar la experiencia del usuario: Al utilizar estas herramientas, podemos ofrecer una experiencia más fluida y personalizada al usuario, ya que podemos "recordar" sus preferencias y estados previos, como el idioma o el tema seleccionado, sin necesidad de que vuelva a configurarlos.

Reducir la dependencia del servidor: Almacenar datos en el navegador permite reducir la cantidad de llamadas al servidor, lo que mejora el rendimiento de la aplicación. Por ejemplo, si un usuario selecciona un tema oscuro, no es necesario hacer una llamada al servidor para recordarlo en futuras visitas; Local Storage puede manejar este tipo de persistencia.

Transferencia eficiente de datos: JSON nos permite manejar datos estructurados de manera eficiente y sencilla en JavaScript. Gracias a su ligereza y universalidad, es perfecto para intercambiar datos entre el cliente y el servidor sin sobrecargar la aplicación.

Ejercicio práctico: Aplicación web que recuerda preferencias de usuario

Imaginemos una aplicación web en la que el usuario puede seleccionar un tema (claro u oscuro) y guardar su nombre. Queremos que cuando el usuario vuelva a visitar el sitio, estas preferencias se recuerden automáticamente, mejorando la experiencia de uso.

HTML:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Preferencias de Usuario</title>
  <style>
    body.oscuro {
      background-color: black;
      color: white;
    }
  </style>
</head>
<body>
  <h1>Bienvenido a la aplicación</h1>

  <label for="nombre">Nombre:</label>
  <input type="text" id="nombre" placeholder="Ingresa tu nombre">

  <label for="tema">Tema:</label>
  <select id="tema">
    <option value="claro">Claro</option>
    <option value="oscuro">Oscuro</option>
  </select>

  <button id="guardar">Guardar Preferencias</button>

  <script src="app.js"></script>
</body>
</html>
```

JavaScript (app.js):

```
// Al cargar la página, recuperar las preferencias del usuario
window.onload = () => {
  const preferenciasGuardadas =
    localStorage.getItem('preferencias');
  if (preferenciasGuardadas) {
    const preferencias = JSON.parse(preferenciasGuardadas);
    document.getElementById('nombre').value =
      preferencias.nombre;
    document.getElementById('tema').value = preferencias.tema;

    // Aplicar el tema guardado
    if (preferencias.tema === 'oscuro') {
      document.body.classList.add('oscuro');
    }
  }
};

// Guardar las preferencias del usuario en Local Storage
document.getElementById('guardar').addEventListener('click', () => {
  const nombre = document.getElementById('nombre').value;
  const tema = document.getElementById('tema').value;

  const preferencias = {
    nombre: nombre,
    tema: tema
  };

  localStorage.setItem('preferencias',
    JSON.stringify(preferencias));
  alert('Preferencias guardadas');

  // Aplicar el tema seleccionado
  if (tema === 'oscuro') {
    document.body.classList.add('oscuro');
  } else {
    document.body.classList.remove('oscuro');
  }
});
```



```
});
```

Este código muestra cómo utilizar Local Storage para recordar las preferencias de un usuario, como su nombre y tema preferido, y cómo JSON permite manejar esos datos complejos de forma estructurada.

Conclusión

Local Storage, Session Storage y JSON son herramientas fundamentales para manejar la persistencia de datos en el navegador. Nos permiten mejorar la experiencia del usuario al "recordar" sus preferencias, aligerar la carga en el servidor y transferir datos de manera eficiente entre sistemas. En conjunto, hacen que las aplicaciones web sean más fluidas, rápidas y personalizadas para cada usuario.